

★ Member-only story

Karpathy's LLM Wiki: How to Actually Use AI So It Stops Starting Over



Kristopher Dunham

Follow

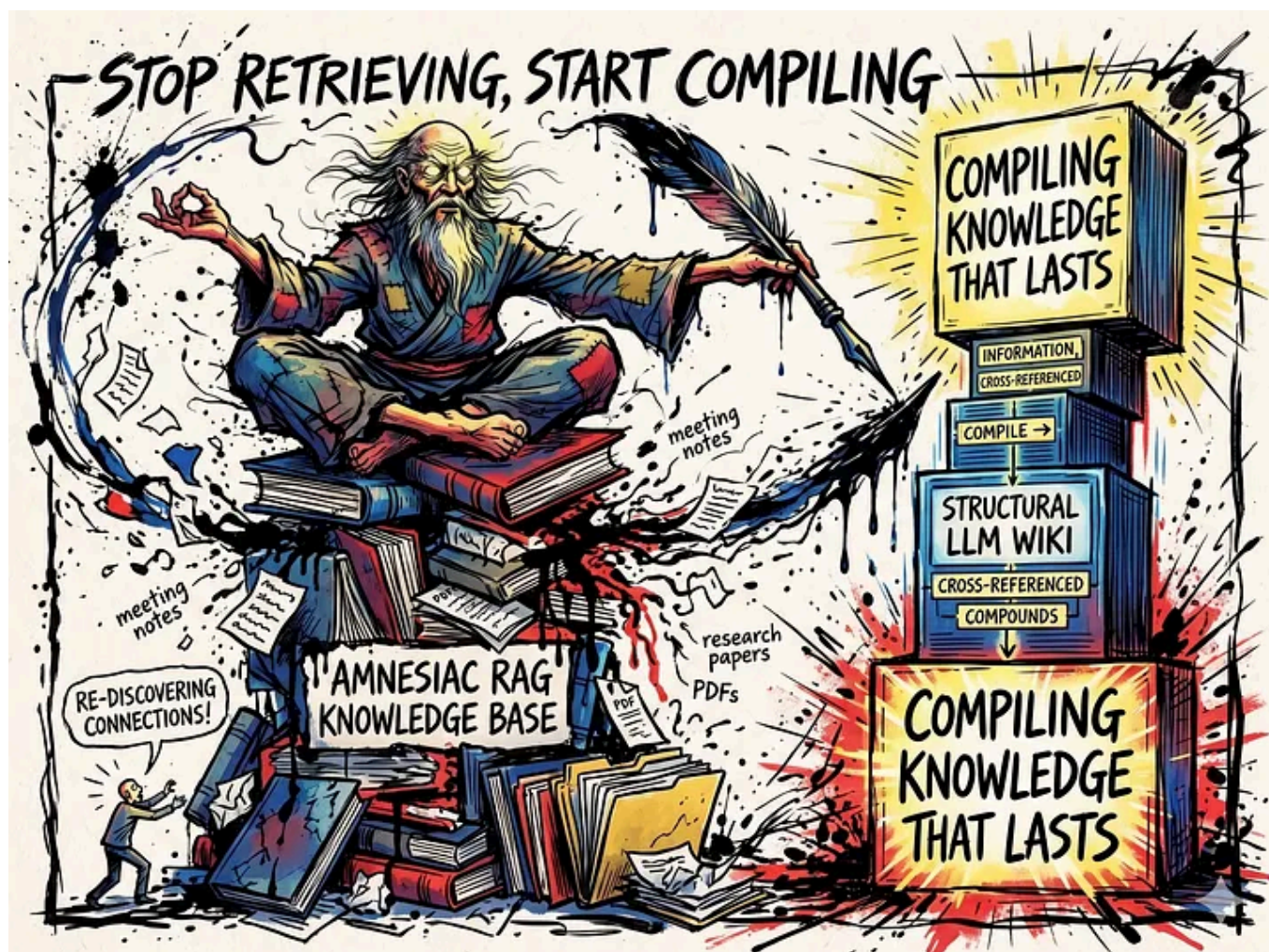
10 min read · Apr 21, 2026



250



2



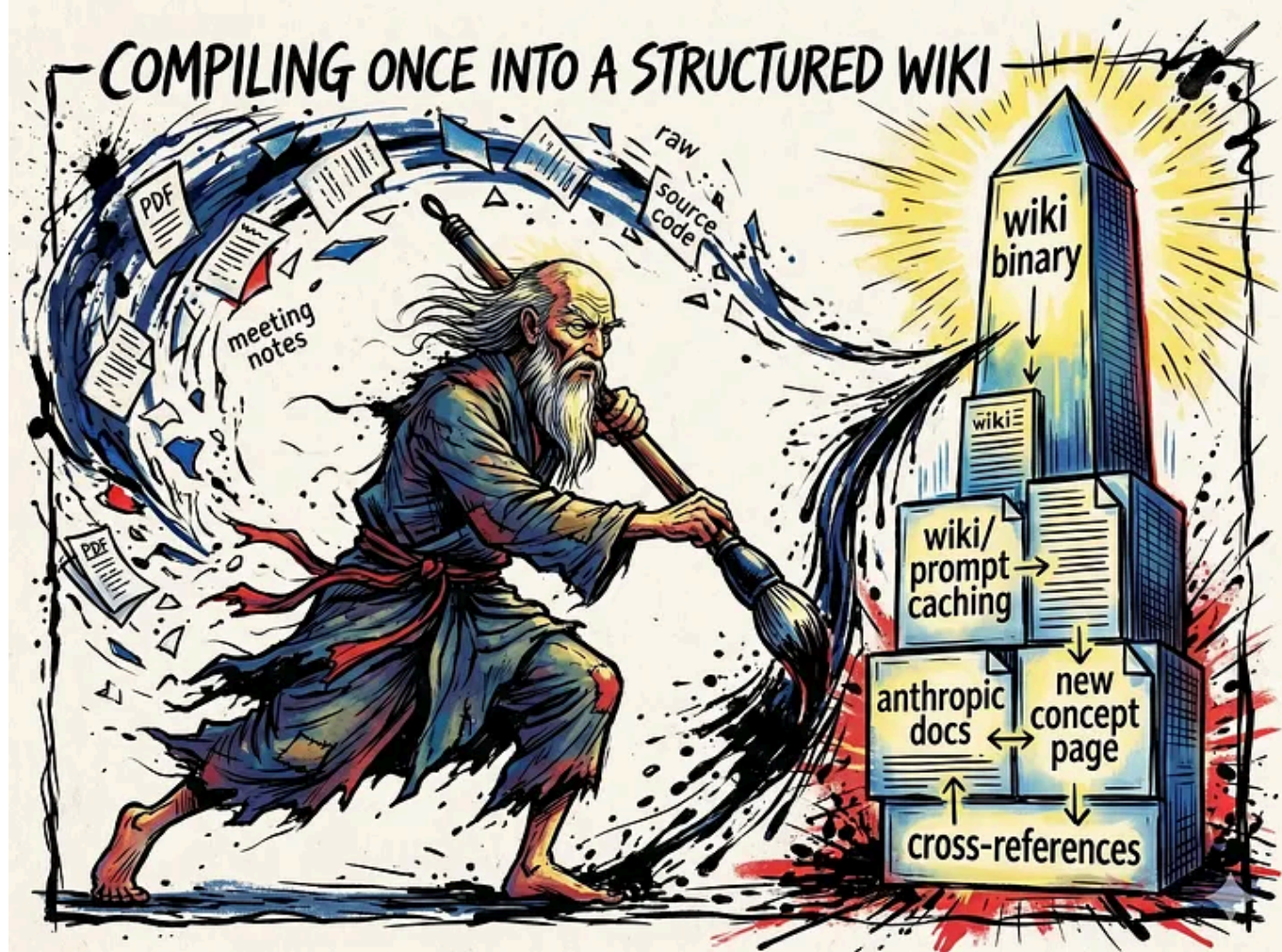
Every AI knowledge tool you've used has the same quiet problem. You upload a folder of PDFs, meeting notes, and research papers. You ask a smart question that requires stitching together five different sources. The model does the synthesis, gives you a great answer, and then forgets it ever happened. Tomorrow you ask the same question. It does the same work from scratch. Nothing accumulates. Your AI is brilliant and permanently amnesiac.

The fix is one sentence long: stop making the model *retrieve* knowledge at query time, and start making it *compile* knowledge once into a structured wiki that compounds over time.

That's the LLM Wiki pattern. Andrej Karpathy posted it as a GitHub gist in April 2026. It hit 16 million views and 5,000 stars within days, and the ecosystem of community tools that sprang up around it tells you the idea struck something real. The pattern is simple enough to build in a weekend, and it's one of the most practical changes you can make to how you work with AI right now.

The compiler, not the search engine

Here's the mental shift. Treat your raw documents the way a compiler treats source code.



In a RAG setup, the model is a search engine. You feed it a question, it does a vector search, grabs the top five chunks, and generates an answer from whatever fragments it pulled. Narrative coherence gets shredded during chunking. A conditional statement on page 14 gets separated from the claim it qualifies on page 13. The model confidently answers without the caveat. You never know what got cut.

In a wiki setup, the model is a compiler. When you drop a new source into the system, it reads the whole thing, then updates existing wiki pages with new claims, creates new pages for new concepts, and stitches cross-references between them. By the time you actually ask a question, the synthesis work is already done. The wiki page on “prompt caching” already integrates what you learned from the Anthropic docs, the paper you read last month, and the Hacker News thread from yesterday.

The model isn't rediscovering connections. It's reading the pre-compiled answer.

Karpathy's own framing is direct: raw sources are source code, the LLM is the compiler, the wiki is the compiled binary. Queries run against the binary. Linting operations act as CI/CD for the knowledge graph.

That analogy does more than sound clever. It tells you exactly how to structure the system.

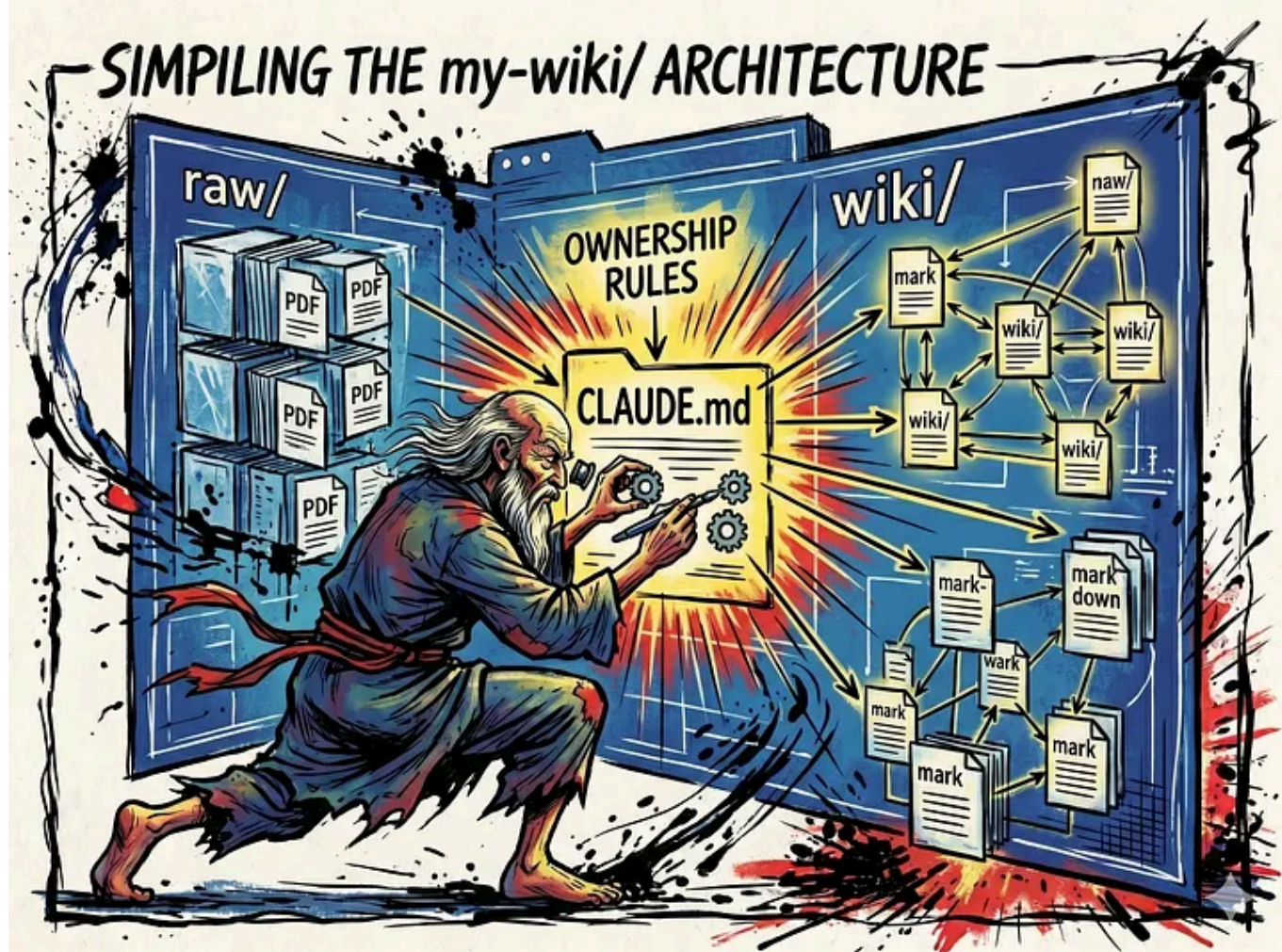
The three layers

There are three folders, and they have strict ownership rules. Break the rules and the system falls apart.

raw/ is immutable. This is where original sources live: PDFs, YouTube transcripts, Markdown exports of web articles, meeting notes, code repos. The LLM never edits these. You rarely edit them either. They're the ground truth. If the compiled wiki ever gets corrupted by a bad ingestion run, you wipe everything and recompile from raw/.

wiki/ is the LLM's domain. This is a dense collection of plain Markdown files, one file per concept, with YAML frontmatter that tracks creation date, tags, and provenance (which raw sources contributed to this page). Pages link to each other using `[[slug]]` syntax, the same format Obsidian uses. There are two special files: `index.md`, which is the auto-generated table of contents the LLM reads first to navigate the wiki, and `log.md`, an append-only ledger of every operation performed. **The hard rule: humans read the wiki, humans don't write to the wiki.** Manual edits break the LLM's internal mapping of what it wrote and why.

The schema is a single file, usually called `CLAUDE.md` or `AGENTS.md`, that defines the rules. It tells the model what kinds of pages exist, what goes in frontmatter, when to create a new page versus update an existing one, and what your domain's conventions are. This file is how you stay in control. You don't edit wiki pages; you edit the schema, and the next ingestion pass applies the new rules.



That's the whole architecture. Three folders and a ruleset.

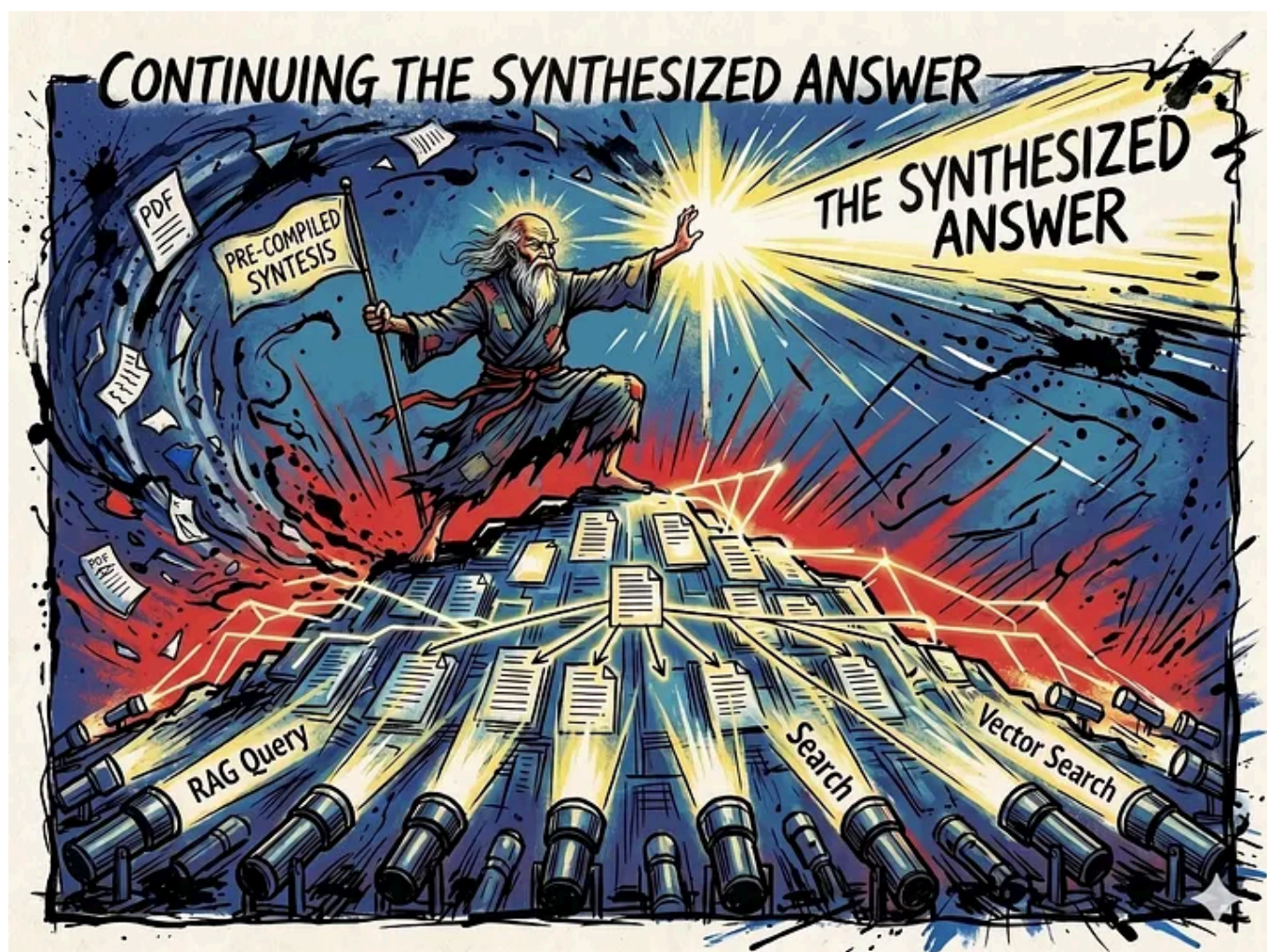
The three operations

There are also exactly three operations the system performs, and getting them right matters more than any tooling choice.

Ingest. You drop a new source into `raw/` and the agent processes it. It reads the source, consults `index.md` to figure out which existing wiki pages are relevant, updates those pages with new claims, creates new pages for concepts that didn't exist yet, flags contradictions (it doesn't silently overwrite old claims with new ones, it marks the conflict in the text with both sources cited), and appends a receipt to `log.md`. A single research paper might touch 10-15 wiki pages in one ingestion pass.

Query. You ask the wiki a question. The agent reads `index.md`, follows the `[[slug]]` links to relevant pages, and synthesizes an answer from the pre-compiled knowledge. Because the pages already contain integrated insight

rather than raw chunks, the model isn't doing synthesis under pressure. It's reading an encyclopedia entry you already wrote together.



There's a subtle but critical feature here called the `--save` flag. When you ask the wiki a question and the answer represents genuine new insight (a comparison you asked for, a connection between two domains), you can tell the model to file that answer back into the wiki as a new page. Now your next session benefits from the thinking you just did. Your explorations compound alongside your ingested sources.

Lint. Periodically you ask the model to health-check the wiki. Standard checks look for orphan pages, broken `[[slug]]` references, and missing concepts (slugs mentioned in text but lacking their own page). Deep checks turn the model into an adversarial fact-checker: find contradictions, find stale claims superseded by newer sources, find circular reasoning. The lint report either gets resolved manually or with an automated `--fix` pass.

That's it. Three folders, three operations. The simplicity is the point.

How to actually build this

The fastest path to a working wiki uses three free tools: **Obsidian** as your viewer, **Claude Code** as your maintainer agent, and (optionally) **QMD** as a search layer once you scale past a few hundred pages.

Start with this directory structure:

```
my-wiki/
├── raw/
│   ├── articles/
│   ├── papers/
│   └── notes/
├── wiki/
│   ├── index.md
│   └── log.md
└── CLAUDE.md
```

Copy Karpathy's gist into `CLAUDE.md` as your starting schema. (The gist is at gist.github.com/karpathy/442a6bf555914893e9891c11519de94f, and it's designed to be pasted directly into your agent as an idea file.) Point Claude Code at the directory. Drop your first source into `raw/articles/`. Tell Claude: "read `CLAUDE.md` then ingest the new file in `raw/articles/`."

That's your first ingestion. Look at what Claude created in `wiki/`. You'll almost certainly want to refine the schema based on what it produced. Maybe you need different page types. Maybe your frontmatter needs a source-date field. Edit `CLAUDE.md`, not the generated pages. Karpathy's phrase is that you and the LLM co-evolve the schema over time. The first version will be rough.

After ten or twenty sources, start querying. Ask questions that require synthesis across multiple pages, not simple lookup. If the wiki gives you an

insight you wouldn't have gotten by reading the sources individually, the system is working. Scale from there.

Run lint weekly. Don't skip this. It's the maintenance loop that keeps the system trustworthy after the novelty wears off.

If you prefer a pre-built implementation over rolling your own, there are several solid community projects: `ussumant/llm-wiki-compiler` is a Claude Code plugin that handles compile, query, and visualize commands.

`Ar9av/obsidian-wiki` wraps the pattern as Agent Skills that work with Claude Code, Cursor, or Codex. `lucasastorian/llmwiki` is an open-source web app where you upload documents and connect your Claude account via MCP.

Where this stops working

This isn't a silver bullet. The pattern was explicitly designed for personal and team-scale knowledge, not enterprise-scale. There are four places it breaks, and you should know them before you commit.

The context window is no longer the ceiling it was. When Karpathy published the gist, the pattern was bounded by 200K-token models. That ceiling is gone.

As of April 2026, **Claude Opus 4.7** and **Gemini 3.1 Pro** both ship with 1M-token input context windows, and Opus 4.7 runs the full million at standard API pricing with no long-context premium. The math is tighter than most writeups admit, though: Opus 4.7 uses a new tokenizer that produces up to 35% more tokens per character than Opus 4.6, so 1M tokens covers roughly 555,000 words of English, not the 750,000 you'll see quoted from older models.

Call it 300 to 400 densely written wiki pages with room left over for the agent's reasoning and output. For most personal or team-scale wikis, you will never hit this ceiling. **QMD** (Tobi Lütke's hybrid BM25/vector search tool, recommended by Karpathy for large wikis) gets pushed further down the

road than it used to be. If you do cross 1M tokens, you're back to retrieval over your compiled wiki pages, and the wiki becomes a stateful compilation layer above RAG rather than a replacement for it.

Ingestion is still expensive, but the economics have shifted. A stateless RAG pipeline chunks a paper, embeds it, and stores the vectors. Cheap. An LLM Wiki ingestion reads the paper, reads the index, reads the 10 to 15 existing pages the new source might touch, then rewrites all of them with the new claims integrated. That's a lot of input and output tokens per source.

Two things make this more livable than it was six months ago. First, Opus 4.7's adaptive thinking means the model allocates its own reasoning budget per step rather than burning a fixed thinking budget on every call, which cuts waste on routine synthesis. Second, prompt caching hits at roughly 10% of the standard input rate, so caching your schema file, index, and stable wiki pages brings effective ingestion cost down sharply across a batch.

Plan for ingestion as the dominant cost bucket, but the bill is smaller than it was, especially if you're running batches of sources overnight through the Batch API for another 50% off.

Hallucinations can get baked in. This is the failure mode critics have been loudest about. In traditional RAG, if the model hallucinates, the damage is contained to that chat session. Your vector database stays pristine.



In an LLM Wiki, the model has write access to the ground truth. If it misreads a paper during ingestion and writes a false causal claim into a wiki page, that falsehood can become the foundation for every future query on the topic.

The good news: this is actually easier to fix here than in most AI systems. Because the pattern mandates provenance tracking (every wiki claim hard-links back to a specific file in `raw/`), tracing a suspect claim to its source takes seconds, not hours. Implementations like `Ar9av/obsidian-wiki` go further with delta manifests that let you roll back specific ingestions. Compare that to untangling a bad chunk from a vector database, where you often can't tell why the model said what it said.

The caveat: the mitigation only works if you actually use it. If you never lint, never check provenance, and never segregate experimental ingestions from trusted ones, minor errors compound silently over months. The tooling gives you the audit trail; the discipline to use it is on you.

The critics aren't wrong about the bookkeeping. The harshest critique I've seen is from a Hacker News user who pointed out that the bookkeeping Karpathy is outsourcing (filing, cross-referencing, summarizing) is *exactly where human understanding is formed*. When you manually write a summary

of a paper into your own notes, the act of writing it is the learning. Hand that off to an LLM and you end up with a beautifully organized wiki that you don't actually understand. This concern is real, and the honest answer is that the LLM Wiki is better for knowledge you've already processed and want to retain, or knowledge you need operational access to without needing to learn deeply. It's worse for knowledge where the filing itself is the point.

Karpathy himself, notably, has warned that vibe-coded software grows beyond the author's ability to understand it. The same critique applies to a vibe-compiled wiki. Use the pattern accordingly.

Who this is for

This pattern is a strong fit if you're a researcher building a thesis over months and want your accumulated reading to stay queryable, a developer who wants your coding agent to remember architectural decisions across sessions (Ar9av's project does exactly this), or anyone who has a pile of articles, papers, or notes and has never been able to make them actually useful.

It's a poor fit if your corpus is measured in millions of documents, if freshness in the last 10 seconds matters, or if you need the act of filing to be the act of learning.

For everyone in between, the five-minute setup is worth the afternoon. Start with ten sources on one topic. Run one ingestion. Ask one synthesis question. If the wiki surfaces something you wouldn't have found otherwise, keep going. If it doesn't, you've lost an afternoon and learned where the pattern's edges are.

Either outcome beats pretending your RAG pipeline will eventually start remembering things.



Written by Kristopher Dunham

711 followers · 9 following

Follow

Published author, developer, creative who builds worlds. Fervorlife.com. Join the fun at CreativeAiDojo.com or CreativeAi.Ninja

Responses (2)



Christian Broberg

What are your thoughts?

See all responses

